18 min read

Model-Free Learning

RL Part 4: Learning value functions and policies without a model. Monte Carlo methods, TD(0), SARSA, Q-learning, and the bias-variance bridge between them.

Recap

In the previous chapter, we explored the recursive structure that sits at the heart of reinforcement learning: the Bellman equations.

We started with the Bellman expectation equations for v_π and q_π. We saw that the value of a state, under a fixed policy, equals the expected immediate reward plus the discounted value of the next state. The equations gave us a way to characterize v_π and q_π exactly, without simulation.

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \Big[R(s,a,s') + \gamma \, v_\pi(s') \Big]$$

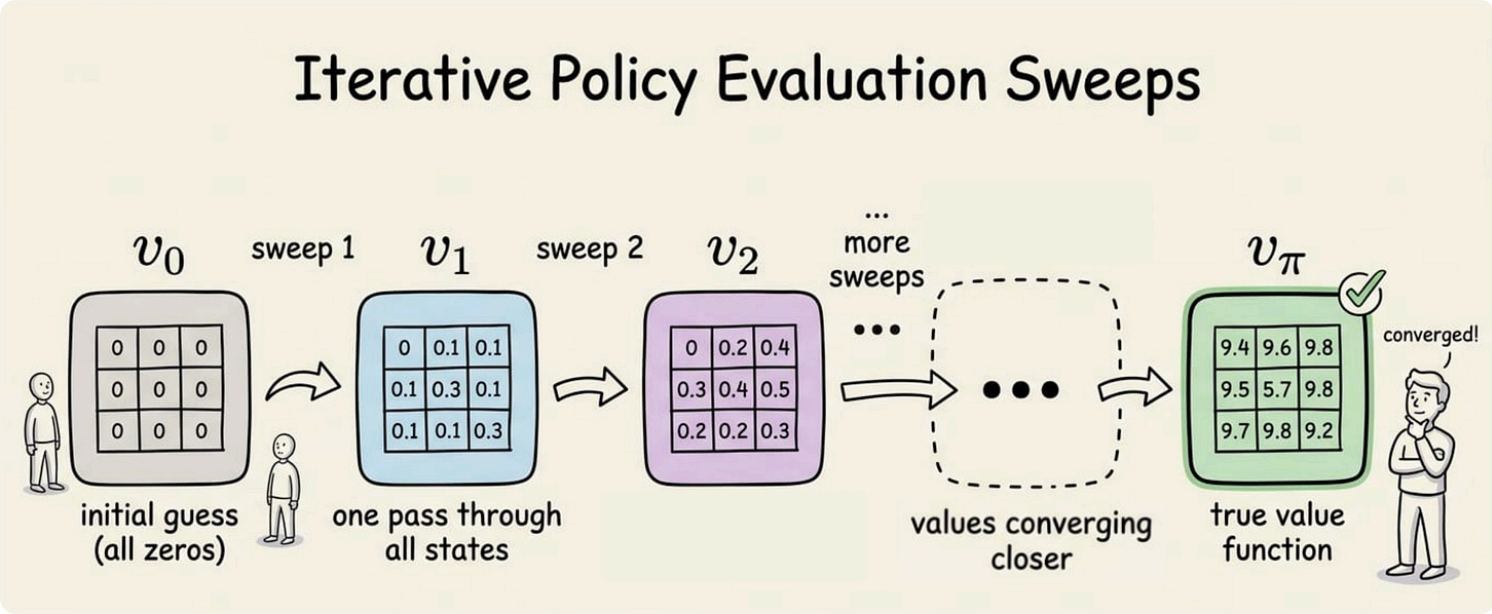
$$q_\pi(s,a) = \sum_{s'} P(s'|s,a) \Big[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') \, q_\pi(s',a') \Big]$$

We then derived the Bellman optimality equations for v_* and q_*. These replaced the policy-weighted average with a max over actions, characterizing the best achievable performance in the environment.

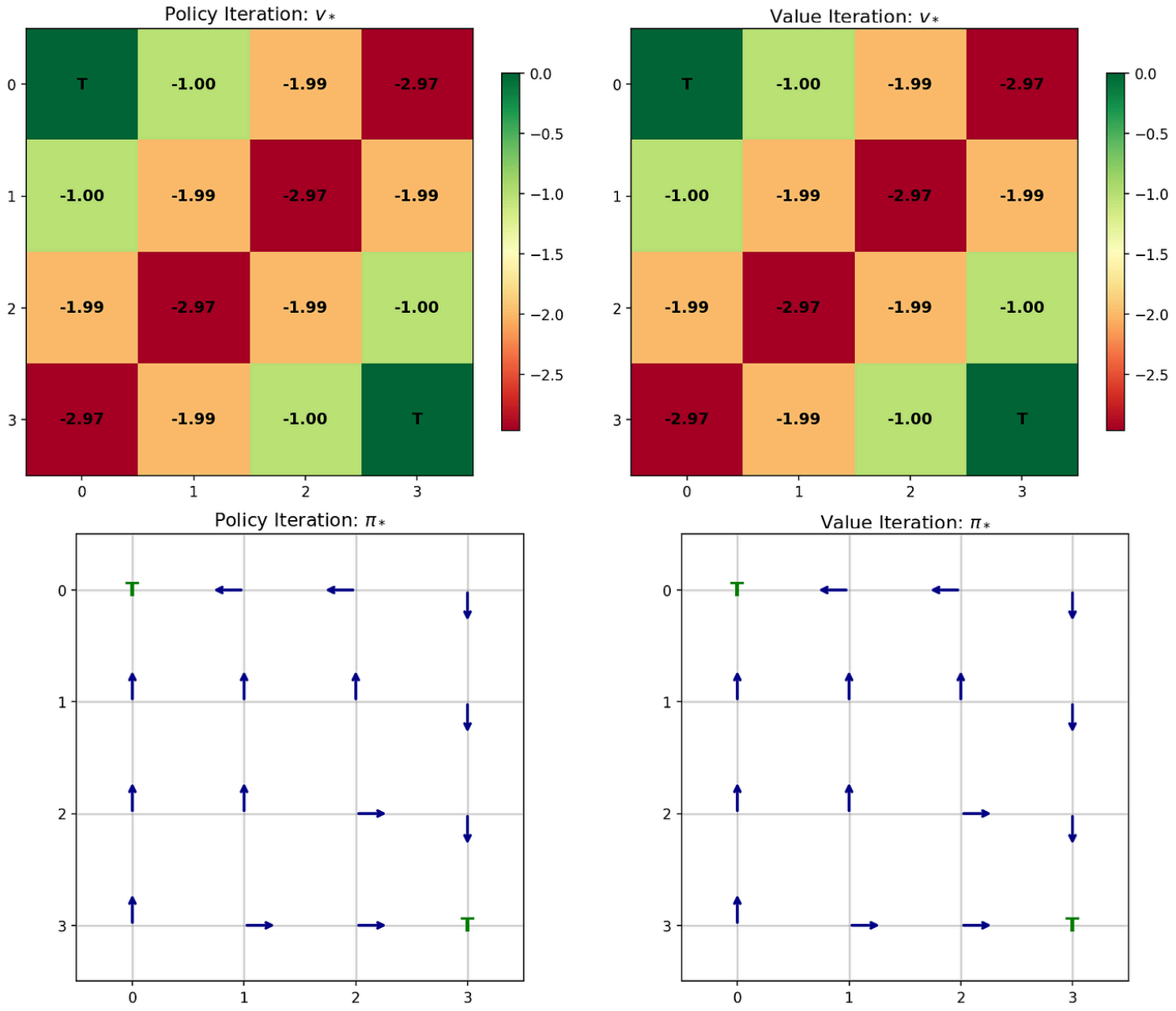
$$v_*(s) = \max_\pi v_\pi(s) \quad \text{for all } s \in S$$

$$q_*(s,a) = \max_\pi q_\pi(s,a) \quad \text{for all } s \in S, \, a \in A$$

We then turned these equations into algorithms via dynamic programming (DP): policy evaluation, policy improvement, policy iteration, and value iteration.



Finally, we ran both policy iteration and value iteration on a 4×4 gridworld and confirmed they converged to the same optimal policy.

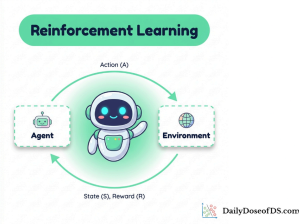


If you have not read Chapter 3, we recommend doing so first:

Bellman Equations and Dynamic Programming

RL Part 3: Bellman expectation and optimality equations, policy iteration, value iteration, and why dynamic programming needs a model.

Daily Dose of Data Science • Avi Chawla



Introduction

DP gave us the gold standard for small, fully known MDPs. The catch is that real environments rarely hand us a clean P and R.

What we usually do have is something else: an agent that can interact with the environment, take actions, and observe rewards and next states. The question for this chapter is how to estimate value functions and learn good policies purely from this kind of experience, without any access to P or R.

This is the model-free setting.

We will look at two foundational families:

- Monte Carlo (MC) methods, which learn from full episodes of experience.
- Temporal-difference (TD) methods, which learn from single transitions by using their current estimates to update themselves.

From there, we will move to TD-based control: SARSA and Q-learning, and close with an experiment that contrasts the two.

Let's begin!

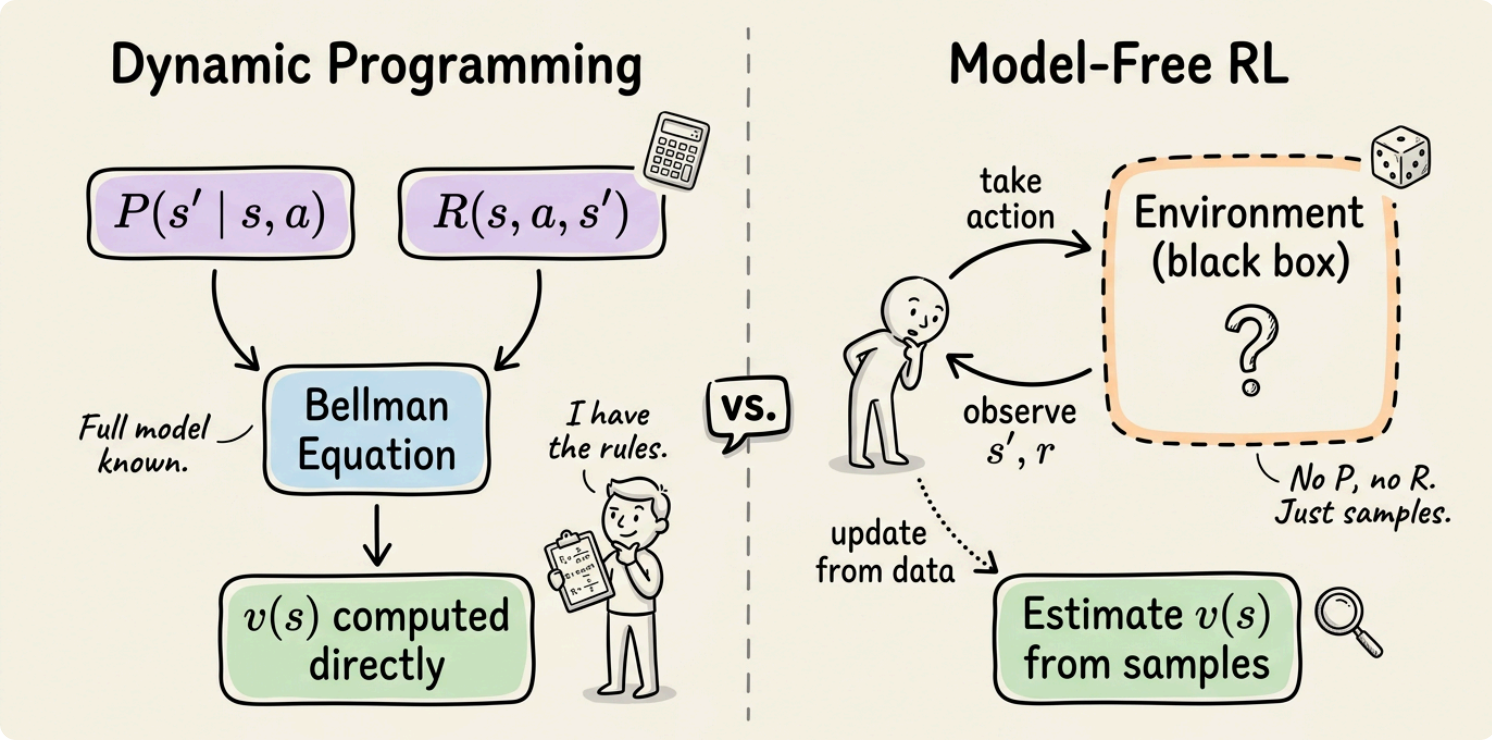
What model-free actually means?

Although we briefly introduced the idea of model-free learning in Chapter 2, it is worth revisiting now. A helpful way to contextualize the term is by contrasting it with the methods explored in Chapter 3.

In DP, we plugged P and R directly into the Bellman update. The agent never had to do anything. We computed v_π or v_* by sweeping over the state space and applying the equations. There was no interaction with the environment, no

episodes to play out and no exploration to manage. We treated the environment as a known mathematical object.

In model-free RL, we do not have P or R. We have an agent that can be placed in a state, take an action, and observe the resulting next state and reward. From these samples, we estimate value functions and improve policies. The environment is treated as a black box.



👉 Important: "Model-free" does not mean no model exists. The environment has dynamics, of course. It means the algorithm does not require access to those dynamics.

Two organizing axes will run through the rest of the chapter:

- The first is prediction versus control. Prediction is the task of estimating v_π or q_π for a given fixed policy. Control is the task of finding a good policy.

👉 Most algorithms in this chapter are introduced in their prediction form first because the math is cleaner, then extended to control.

- The second axis is on-policy versus off-policy. An on-policy method learns about the same policy it uses to generate behavior. An off-policy method can learn about one policy (the target policy) while behaving according to another (the behavior policy).

So now, let's dive into understanding the various model-free techniques.

Monte Carlo prediction

We know that the value of a state under a policy is, by definition, the expected return when starting from that state and following the policy:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

If you cannot compute this expectation analytically (because you do not know P), you can still estimate it by sampling, by running many episodes. Every time the agent visits state s, record the return that followed. The average of those returns is your estimate of $v_\pi(s)$.

By the law of large numbers, as the number of visits to s goes to infinity, the sample average converges to the true expected return. No model required.

👉 This means run more episodes, the average tightens. There is nothing more to it than that.

First-visit and every-visit MC

When state s appears more than once in the same episode, we have a choice:

- First-visit MC averages the return only from the first time s is visited in each episode.
- Every-visit MC averages the return from every visit, treating each one as an independent sample.

Both are valid. Both converge to $v_\pi(s)$ as the number of episodes grows. First-visit MC has a longer history of theoretical analysis and every-visit MC tends to be simpler to implement and converges similarly in practice.

> 👉 MC methods are defined for episodic tasks only. The return G_t has to actually be computable, which means the episode has to end. Continuing tasks (no terminal state) cannot be handled directly by basic MC.

Incremental updates

Computing the average naively is wasteful. We can avoid this by maintaining an efficient running mean.

For a state s with visit count $N(s)$ and current estimate $V(s)$, the update after observing a new return G is:

$$V(s) \leftarrow V(s) + \frac{1}{N(s)}\big[G - V(s)\big]$$

Here:

- $V(s)$ is the running estimate
- G is the new return observed for this visit
- $N(s)$ is the number of times s has been visited so far.
- The bracketed term $G - V(s)$ is the prediction error: how much the new sample disagrees with the current estimate. We move the estimate a fraction $1/N(s)$ of the way toward the new sample.

The structure has a clean interpretation. Each new return nudges the estimate toward the truth. The step size $1/N(s)$ shrinks as visits accumulate, so later

samples have less and less effect. In the limit, the estimate stops moving and we have the exact sample mean.

In practice, however, we often replace $1/N(s)$ with a constant step size α:

$$V(s) \leftarrow V(s) + \alpha\big[G - V(s)\big]$$

Typically, $\alpha \in (0, 1]$. This trades exact convergence for the ability to track non-stationary value functions, where older returns become less relevant.

Now, one last note before we move to control. Everything we said about MC prediction for V extends naturally to Q.

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}[G - Q(s,a)]$$

Also, similar to V, we can replace $1/N(s,a)$ with a constant step size α.

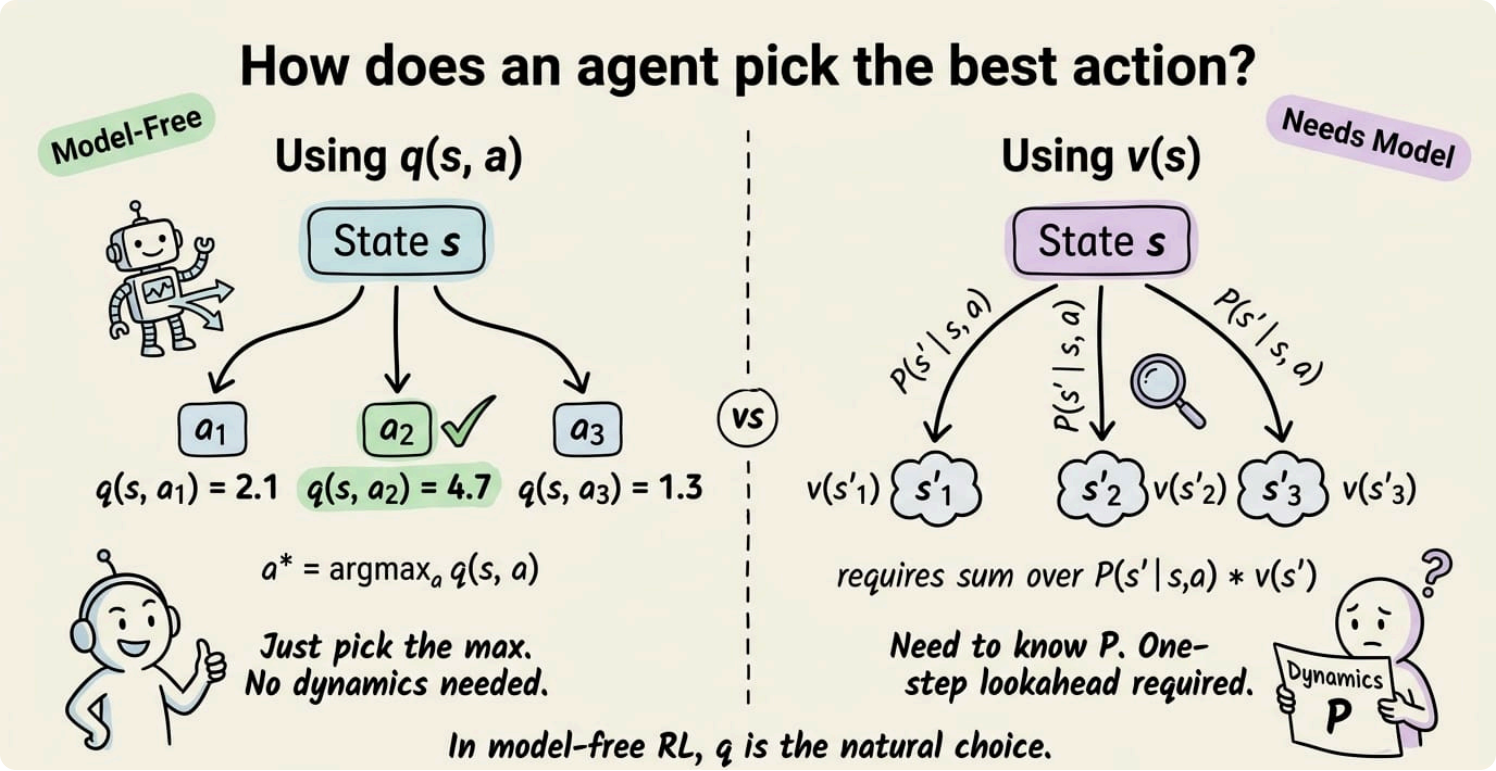
Monte Carlo control

Prediction estimates v_π for a given π. Control is the harder problem: finding a good policy. The natural template, inherited from DP, is the concept of policy iteration: alternate evaluation with improvement.

$$\arg\max_a \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma v_\pi(s')]$$

That requires P and R. Without them, v_π alone is not enough to derive a greedy policy.

But the solution is also something that we already know. Notice that the expression inside the $\arg\max$ is exactly $q_\pi(s,a)$. So the fix is to estimate the action-value function $q_\pi(s,a)$ directly.



Now once we have q_π, the greedy policy is just:

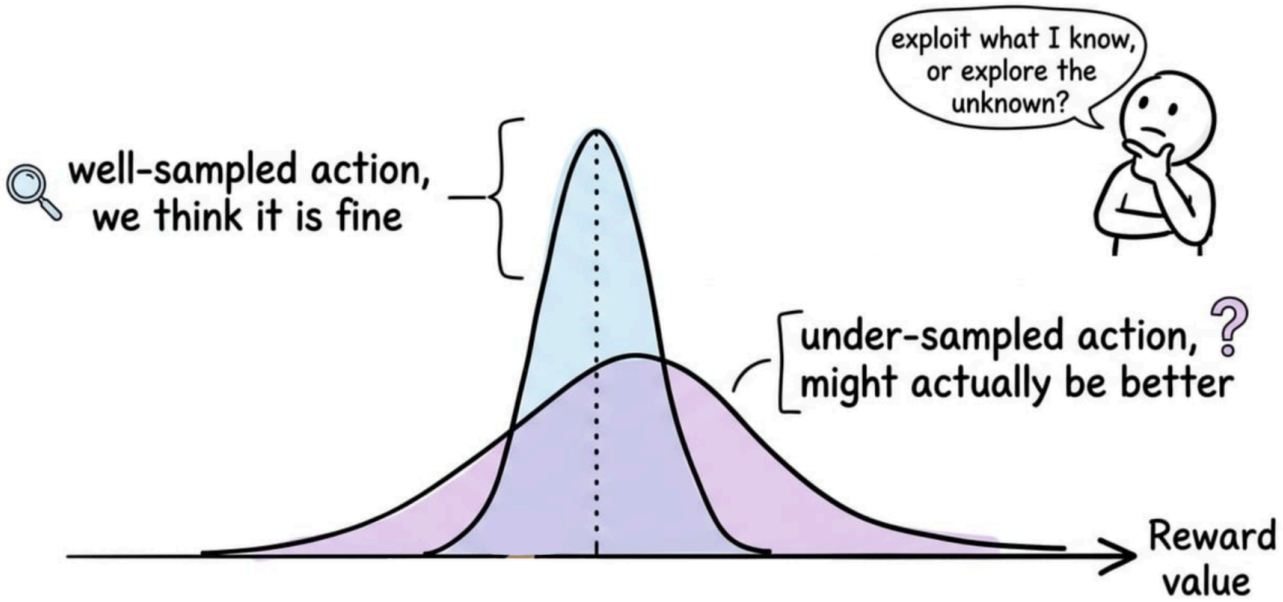
$$\pi'(s) = \arg\max_a q_\pi(s, a)$$

This needs no model and no lookahead, which is why action-value functions dominate model-free RL.

But there is still a snag, the exploration-exploitation misbalance this brings about.

The exploration problem

If we follow a deterministic greedy policy from the start, many state-action pairs will never be visited. Without visits, we never get returns, never update $q_\pi(s, a)$, and never discover that some action might actually be better than what we currently believe.



The greedy policy locks us into whatever happened to be best in our initial estimates, which were arbitrary.

This is the exploration problem in MC control. There are two classical solutions:

- Exploring starts assumes every state-action pair has a non-zero probability of being the start of an episode. Sample the initial state and action uniformly, then follow the policy thereafter. Mathematically clean; but practically, it requires being able to reset the environment to any state-action pair, which is rare in real problems.
- ε-greedy and ε-soft policies keep exploration baked into the policy itself. With probability $1 - \varepsilon$, take the greedy action; with probability ε, take a random action. The policy is "soft" because every action has some probability of being chosen in every state, so all state-action pairs eventually get visited.

👉 The ε-greedy approach is the standard in practice. It is dead simple, requires no special environment access, and gives a tunable knob for the exploration-exploitation trade-off.

Now that we have a reasonable grasp of Monte Carlo methods, let's examine their shortcomings.

Limits of Monte Carlo

MC is conceptually clean but has structural limits:

- Episodic-only: The return G_t is only defined when the episode ends. In tasks that go on indefinitely, MC has no natural target to update toward.
- Wait: Even in episodic tasks, MC has to play out the entire episode before any state's value can be updated. If an episode lasts thousands of steps, every state in it sits stale until the very end.
- Variance: The return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$ accumulates randomness from every step. Two episodes starting from the same state can produce wildly different returns because of the cumulative noise. High variance in the target means many samples are needed before the average settles.

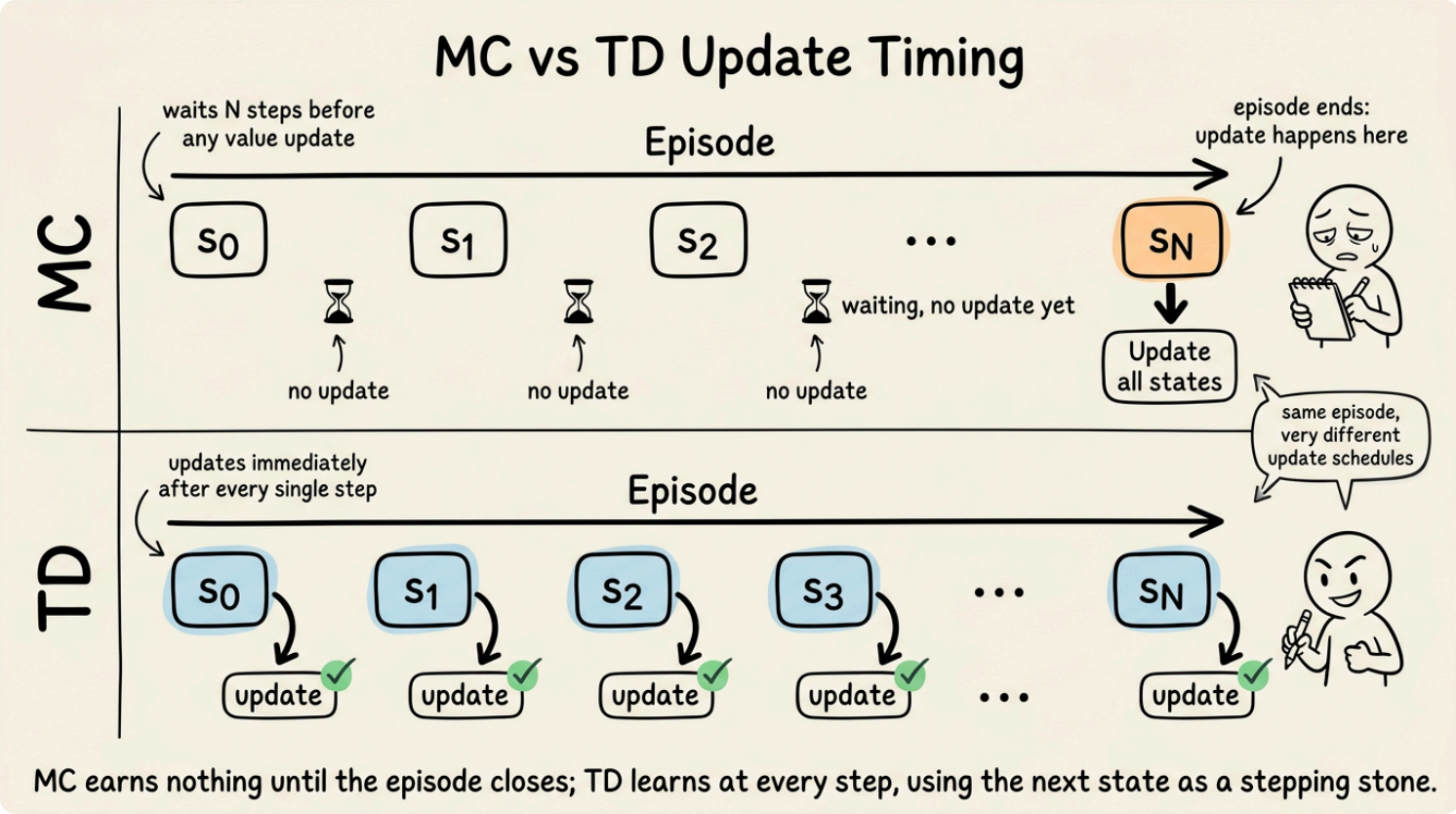
These three problems all have a common root: MC commits to the actual full return as its update target. What if we used a shorter target, one that only goes one step into the future and then bootstraps?

That is exactly the TD idea.

Temporal-difference learning: TD(0)

Temporal-difference learning, introduced by Sutton in his 1988 paper "Learning to Predict by the Methods of Temporal Differences", is the central idea that makes most modern RL work.

The setup is the same as MC prediction: we have a fixed policy π and we want to estimate v_π. The change is in the target. Instead of waiting for the full return G_t, we use a one-step target that combines the immediate reward with the current estimate of the next state's value.



The TD(0) update is:

$$V(S_t) \leftarrow V(S_t) + \alpha\left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]$$

Let's unpack this carefully:

- $V(S_t)$ is our current estimate of the value of the state we just left, S_t.
- R_{t+1} is the reward we observed on the transition.
- $V(S_{t+1})$ is our current estimate of the value of the state we landed in.
- γ is the discount factor.
- α is the step size.

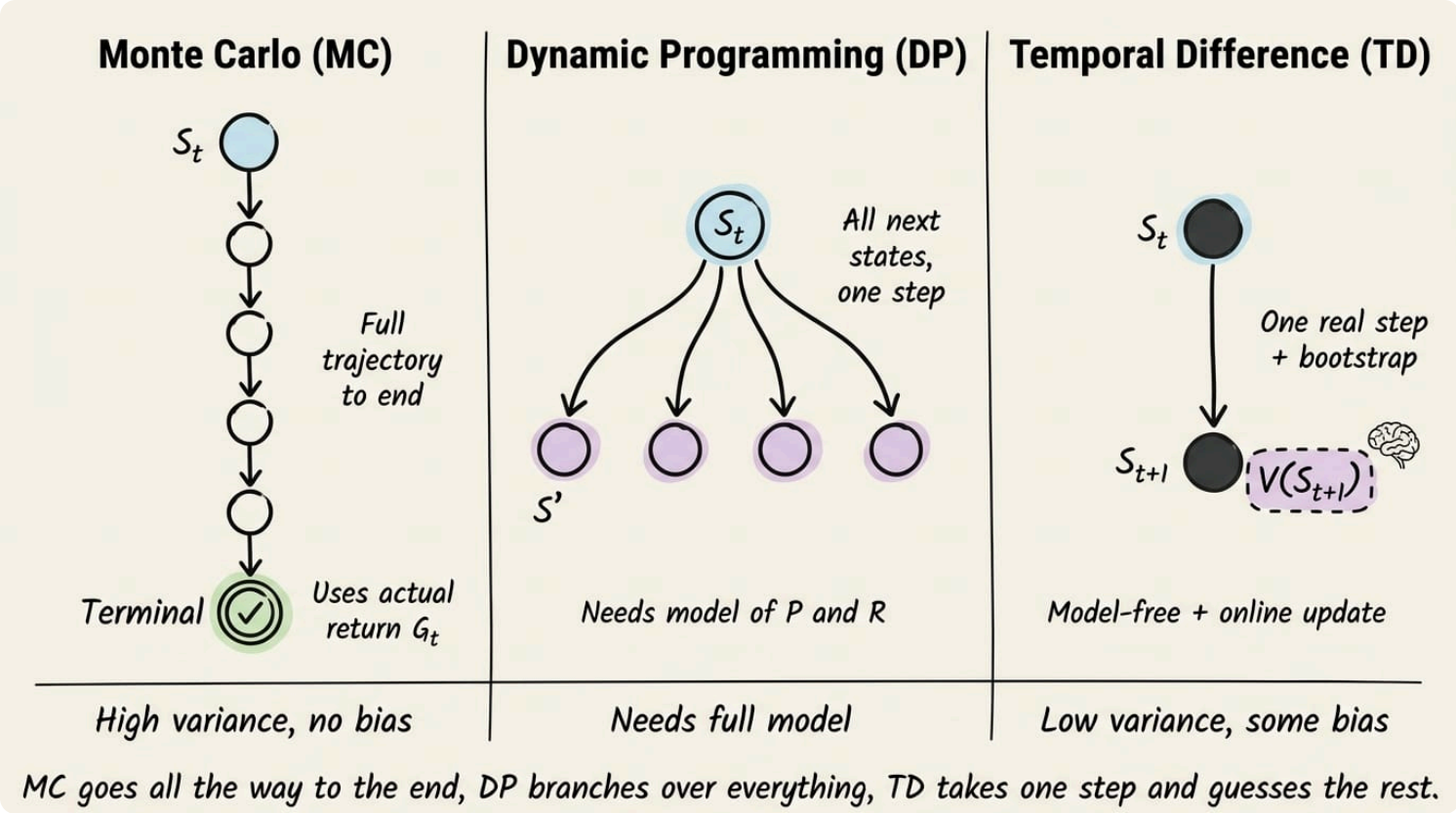
The quantity $R_{t+1} + \gamma V(S_{t+1})$ is called the TD target. It is a one-step approximation of the full return: take the actual reward, then use the value estimate to stand in for the rest of the return after that.

The bracketed term $R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the TD error, often denoted δ_t. It measures how surprising the transition was: how much the new evidence (one step of reward plus the value of where we ended up) disagrees with our current estimate of where we started.

TD as a fusion of MC and DP

TD sits between MC and DP, taking one good idea from each:

- From MC, it inherits the model-free property: the update only uses observed transitions. No knowledge of P or R required.
- From DP, it inherits bootstrapping: the update uses an existing estimate $V(S_{t+1})$ as part of the target. This is what lets TD update online, after every single step, without waiting for the episode to end.



The result is a method that works on continuing tasks and updates online.

Bias and variance: MC vs TD

The MC vs TD comparison is one of the most useful conceptual handles in RL. The difference is best framed in terms of bias and variance of the update target.

MC bias and variance:

- The MC target is the actual return G_t. Because G_t is, by definition, a sample from the true return distribution under π, its expectation is exactly $v_\pi(S_t)$. The MC target is unbiased.
- But G_t is the sum of many random rewards. Each step adds its own noise, and the noise compounds. Two episodes from the same state can yield very different G_t values. The MC target has high variance.

TD bias and variance:

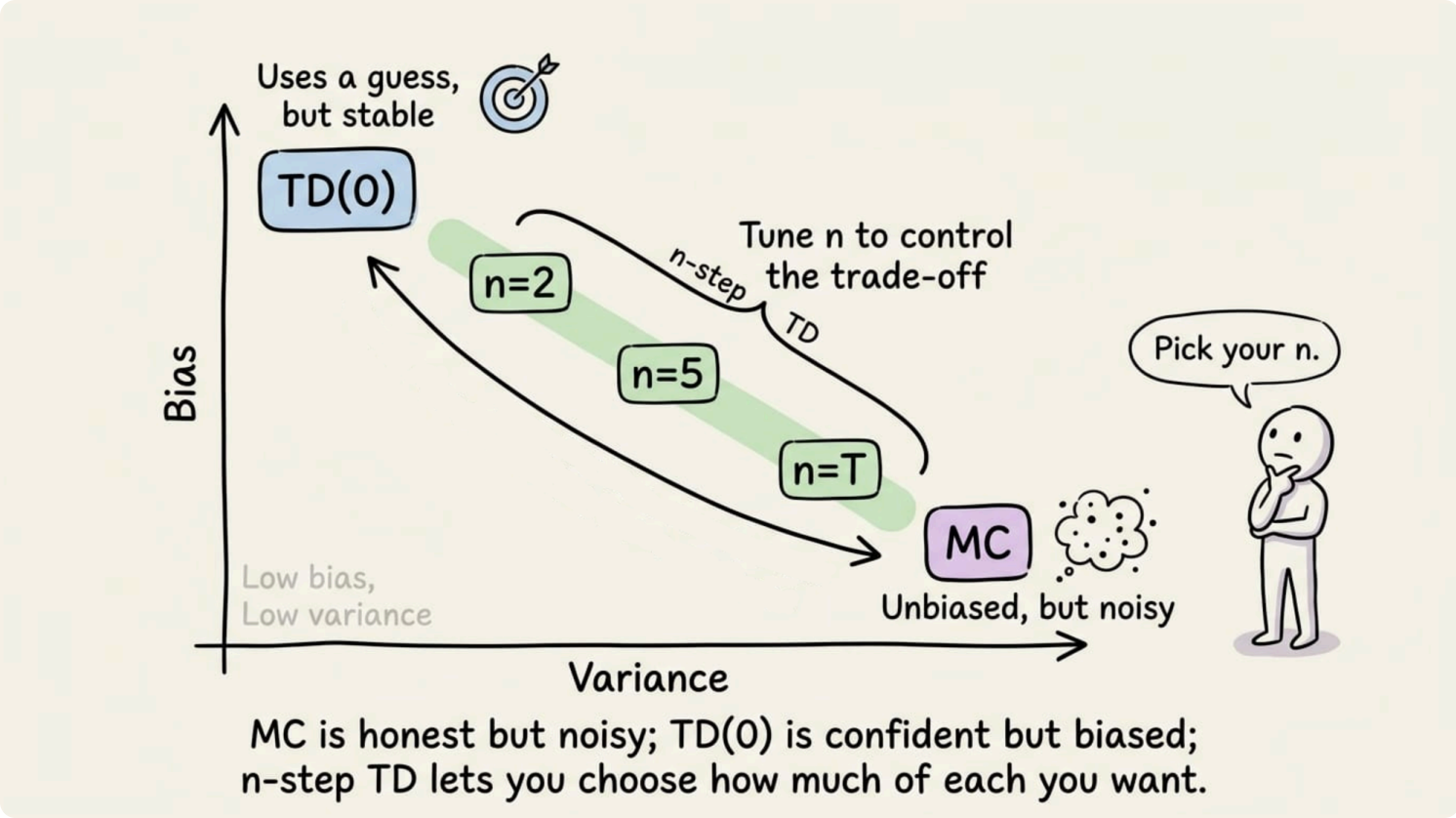
- The TD target is $R_{t+1} + \gamma V(S_{t+1})$. Only one reward enters; the rest of the return is replaced by the current value estimate. Variance is much lower because we are summing over far less randomness.

- But $V(S_{t+1})$ is just an estimate, almost certainly wrong, especially early in training. The TD target uses a guess in place of the truth. The TD target is biased.

So we have a clean trade-off:

- MC: unbiased, high variance.
- TD(0): biased, low variance.

A concrete way to see the variance gap: suppose every reward in an episode is an independent random variable with variance σ^2, and the episode is T steps long. The MC return G_t has variance roughly $T\sigma^2$ (ignoring discounting). The TD target uses one reward and a value estimate, so its variance is roughly σ^2 plus some smaller estimate-related term. For long episodes, that is a difference of an order of magnitude or more.



👉 In practice, on tabular tasks, TD(0) typically converges faster than constant-α MC.

SARSA: on-policy TD control

Before we take a look at SARSA, let's briefly look at what is meant by "on-policy".

The definition is, the policy being learned is the same as the policy generating the data.

> 👉 This in simpler terms means, on-policy means you can only learn from your own decisions. If you didn't make the choice yourself, you can't use it to improve.

Now whatever we saw about TD so far has been a prediction method. To turn it into a control method, we make the same shift we did with MC: estimate action values $Q(s, a)$ instead of state values $V(s)$, and act greedily (or ε-greedily) with respect to Q.

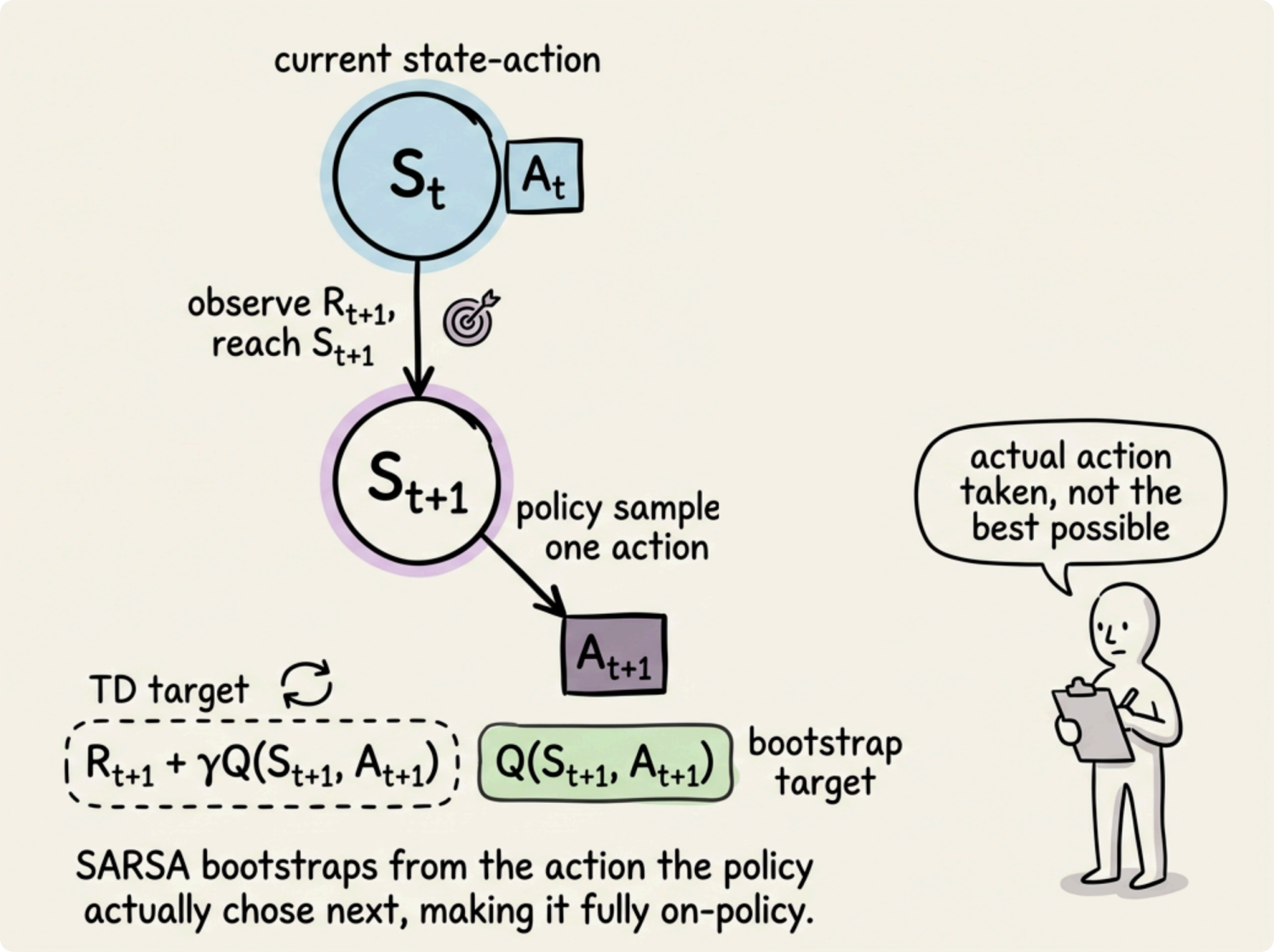
The SARSA update is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$

Here, S_t and A_t are the state and action at time t, R_{t+1} and S_{t+1} are the observed reward and next state, and A_{t+1} is the action selected by the current policy at S_{t+1}.

The structure mirrors TD(0) for V, but every term is now an action-value. The TD target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ uses the value of the actual action that the policy actually took at the next state.

The name "SARSA" comes from the tuple of variables that appears in the update: $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.



The algorithm flow is straightforward:

- Select A_t from the current ε-greedy policy on S_t, take the action.

- Observe R_{t+1} and S_{t+1}, select A_{t+1} from the same ε-greedy policy on S_{t+1}, then apply the update.
- Notice that the update needs A_{t+1} before it runs, this means SARSA picks the next action first and then updates.

> 👉 A subtle point worth flagging: when an episode terminates after the transition from S_t, there is no S_{t+1} to bootstrap from. The standard convention is to treat the value of the terminal state as zero, so the target collapses to just R_{t+1}.

Q-learning: off-policy TD control

Off-policy methods break the coupling between the policy generating data and the policy being learned about. Two distinct policies are involved:

- Behavior policy: the policy the agent actually follows to interact with the environment and generate experience. This is often exploratory, such as an ε-greedy policy.

- Target policy: the policy the agent is trying to learn. This is the policy whose value function ultimately matters, and it is often the greedy policy with respect to Q.

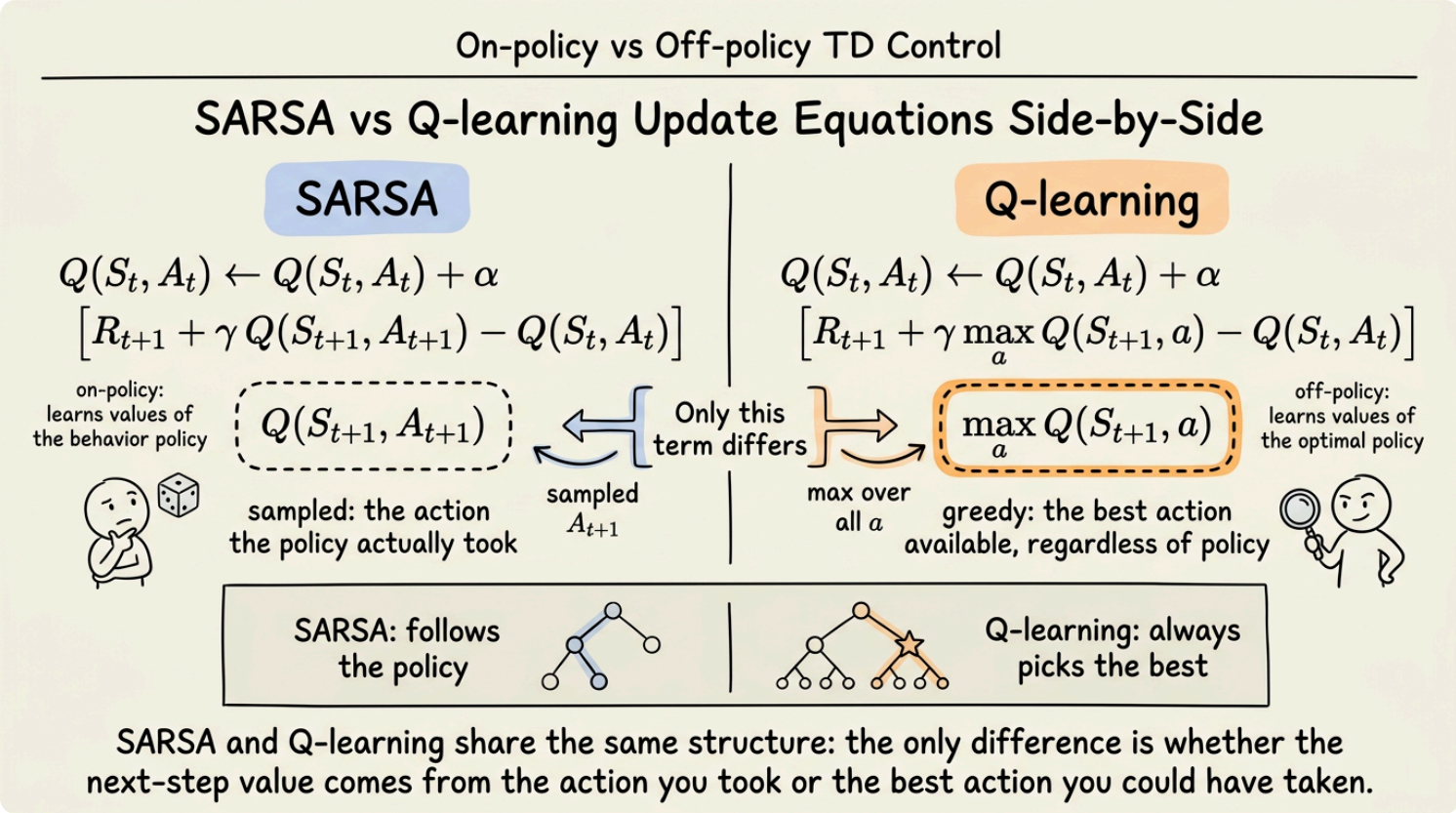
Now coming back to the topic, Q-learning is the pivotal algorithm of classical RL and is the seed of an enormous fraction of modern deep RL.

The Q-learning update is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$

The only change from SARSA is in the bootstrap term. SARSA uses $Q(S_{t+1}, A_{t+1})$, the value of the action the policy actually picks. Q-learning uses $\max_a Q(S_{t+1}, a)$, the value of the best action available at the next state, regardless of what the policy picks.

That single change makes Q-learning off-policy. The update target reflects the greedy (optimal) policy, not the behavior policy used to collect the data. The agent can explore however it likes, and the Q-values it learns will still be approaching q_*.



This is a remarkable property. It means we can run an exploratory ε-greedy behavior policy, collect data, and update toward the values of the optimal greedy policy at the same time.

Now that we know Q-learning's max operator is what gives it off-policy power. It is also what introduces a problem called maximization bias: taking the max over noisy estimates tends to overestimate the true max.

A brief note on maximization bias

Suppose we have several actions whose true Q-values are all equal, but our estimates of those Q-values are noisy. The max of noisy estimates is, in expectation, larger than the max of true values, because the max operator preferentially picks whichever action happened to be overestimated.

The Q-learning target inherits this bias every time it computes $\max_a Q(S_{t+1}, a)$. The bias is small in benign environments and vanishes as estimates tighten, but in stochastic or noisy domains, it can meaningfully delay convergence. SARSA, which uses a sampled action rather than a max, does not suffer from this problem in the same way.

Another practical consequence-based difference in SARSA and Q-learning: if exploration is dangerous, SARSA and Q-learning learn different policies.

> 👉 SARSA's policy is shaped to avoid the danger that the explorative nature of its policy actually creates; Q-learning's policy is the optimal one as if exploration didn't exist, even though the agent is still explores.

For example, imagine teaching a robot to drive a narrow path with a ditch on either side. SARSA's policy would steer toward the middle of the path because, factoring in the robot's noisy steering, the middle is the safest place to be. Q-learning's policy would steer right at the edge of the path because, assuming perfect steering, the edge is fastest. Both policies are correct under their own assumptions. The choice between them depends on whether the noise that the agent uses to explore is also the noise the deployed policy will face.

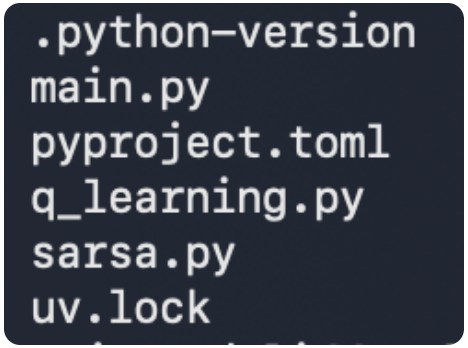
We have now understood all the key concepts for this chapter. The pieces are in place. So let's go ahead and dive into hands-on section for some practical experimentation.

Hands-on: SARSA vs Q-learning on Cliff Walking

We will now walk through a standard experiment for demonstrating the SARSA-vs-Q-learning behavioral difference on a small gridworld called "Cliff Walking".

Setup

The code and project setup are attached below as a zip file. You can extract it and run `uv sync` to get going.



Download the zip file below:

cliff-walking

cliff-walking.zip • 37 KB

For details about versions and dependencies, check the `.python-version` and `pyproject.toml` files.

👉 It is recommended to follow the explanation ahead side-by-side with the above attached project for a more comprehensive understanding.

The environment

Cliff Walking is a 4×12 gridworld. The agent starts at the bottom-left cell. The goal is the bottom-right cell. The bottom row between them is a cliff of 10 cells.

Every step gives a reward of -1. Stepping into the cliff gives -100 and resets the agent to the start (without ending the episode). The episode ends when the agent reaches the goal. There are four actions: up, right, down, and left.

The state space has 48 cells, encoded as flat indices: state $= \text{row} \times 12 + \text{col}$. Start is state 36 (row 3, col 0), goal is state 47 (row 3, col 11), cliff cells are states 37 through 46.

To get a feel for the dynamics, the optimal trajectory under perfect (zero-exploration) play is: from state 36, move UP to state 24, then RIGHT eleven times along row 2 to state 35, then DOWN to state 47. That is 13 steps. Any path that goes higher up the grid (rows 0 or 1) trades off speed for safety: under ε-greedy,

a random action on row 1 is much less likely to plunge the agent off the cliff than the same random action on row 2.

SARSA implementation

SARSA agent and training loop:

The agent stores a Q-table of shape $(48, 4)$, initialized to zero. Action selection is ε-greedy: with probability ε pick a random action, otherwise pick $\arg\max_a Q(s, a)$. The update applies the SARSA rule from earlier in the chapter.

A few details about the implementation:

- The training loop chooses the next action before updating $Q(s, a)$, because SARSA's target needs A_{t+1} from the current policy.
- When the episode terminates, the bootstrap term is dropped: the target is just the observed reward, since there is no successor state.

Q-learning implementation

Q-learning agent and training loop:

The Q-learning agent has the same shape: a Q-table, ε-greedy action selection, the same handling of episode termination. The single meaningful difference is in the update: instead of using $Q(s', a')$ for the next action a' chosen by the policy, it uses $\max_a Q(s', a)$.

In the training loop, this difference shows up in a small but consequential way: Q-learning does not need to know the next action when updating $Q(s, a)$. The action selection happens fresh each loop iteration. SARSA, because it needs A_{t+1}, has to pre-select the next action and carry it through the loop.

Running the experiment

Experiment driver code:

We run both algorithms with identical hyperparameters: $\alpha = 0.5$, $\gamma = 1.0$, $\varepsilon = 0.1$, 500 episodes per run, 10 independent runs per algorithm.

The experiment has three steps:

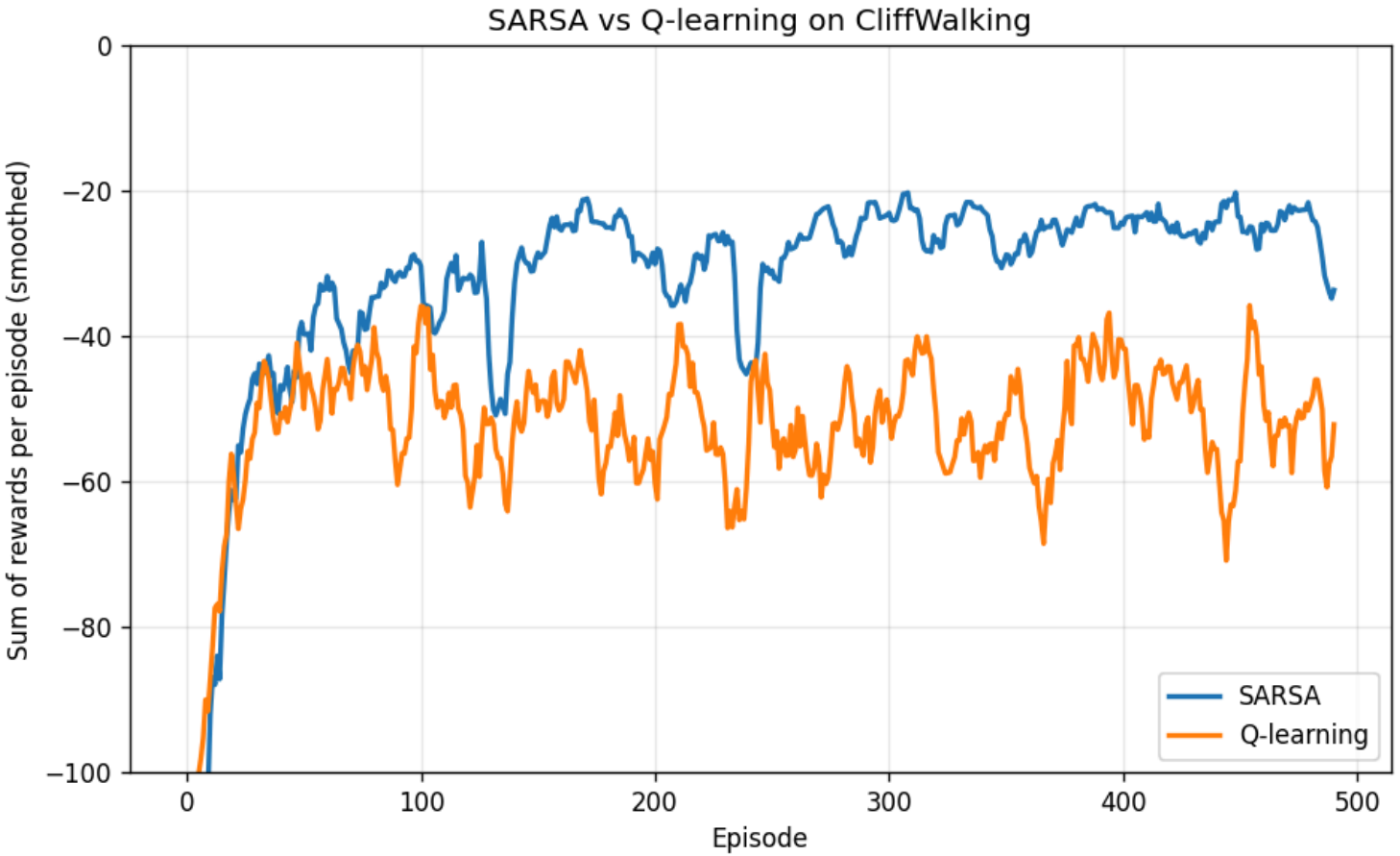
- For each of 10 random seeds, train SARSA and Q-learning from scratch and record the total reward per episode.
- Average the per-episode rewards across the 10 runs for both algorithms, then smooth with a 10-episode moving average for plotting.
- For one representative seed, extract the greedy policies and visualize them on the grid.

Results

Running `python main.py` produces:

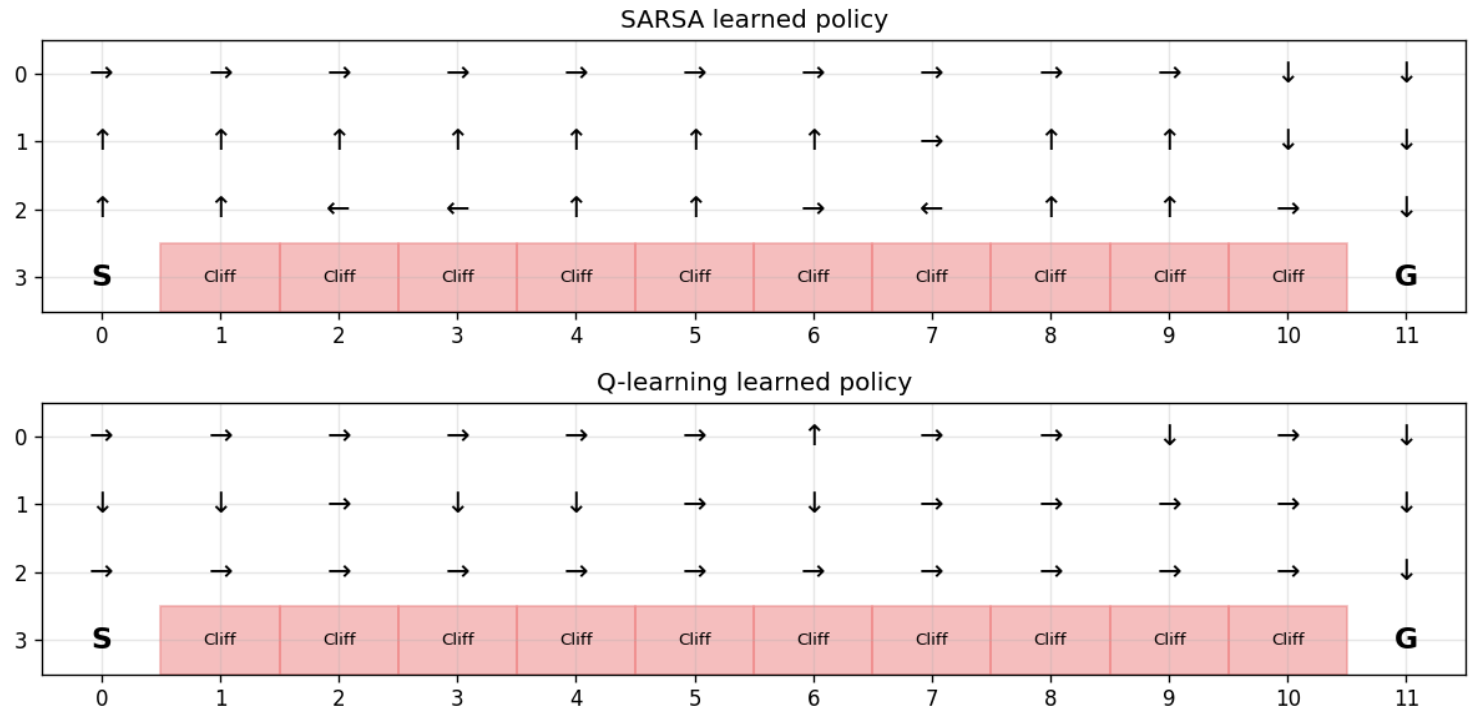
```
Average reward over the last 100 episodes:
  SARSA:      -25.23
  Q-learning: -50.45
```

SARSA accumulates roughly half the negative reward of Q-learning during training. Q-learning falls off the cliff regularly because of ε-greedy exploration, while SARSA learns to stay away from the edge.



The plot above also conveys the same pattern.

Now the policies tell the rest of the story:



SARSA's greedy policy from the start cell goes UP (away from the cliff), then traverses the top of the grid, then DOWN to the goal. The path is longer but safer under ε-greedy: even if exploration takes a random action, the agent is two rows away from the cliff and recovers.

Q-learning's greedy policy from the start cell goes UP one row, then RIGHT all the way along row 2 (the row just above the cliff), then DOWN. This is the optimal path. But under ε-greedy training, the agent is constantly one wrong

action away from falling off, and that is exactly why its training reward is worse despite finding the better policy.

That wraps up the hands-on section. With this, we conclude the discussion for this chapter. In the upcoming chapters, we will continue to build on the core ideas of RL and explore concepts and their implementations wherever applicable.

Conclusion

In this chapter, we explored model-free learning: how to estimate values and learn policies without access to the environment dynamics.

We started by clarifying what model-free actually means: not the absence of dynamics, but the absence of access to them in the algorithm. We laid out two organizing axes: prediction versus control, and on-policy versus off-policy.

We covered Monte Carlo prediction and saw the first-visit and every-visit variants, the incremental update, and the constant-α generalization. We then

extended MC to control: shifting from V to Q, handling exploration via ε-greedy.

We saw the structural limits of MC: episodic-only, must wait for episode end, high variance per update. These motivated TD(0) learning, where the update target combines one observed reward with a bootstrapped value estimate of the next state.

From TD prediction, we moved to TD control:

- SARSA is on-policy and uses the value of the actually taken next action in its target.
- Q-learning is off-policy and uses the max over next actions.

Finally, the Cliff Walking experiment made the on-policy / off-policy distinction concrete:

- Q-learning learned the optimal path along the cliff edge but suffered more during training because of ε-greedy exploration.
- SARSA learned the safer path along the top of the grid and accumulated less negative reward during training.

In the next chapter, we will go beyond tabular methods. The Q-tables and V-tables we have used only work when state and action spaces are small enough to enumerate. For large or continuous state spaces, we need function approximation. That is the bridge from the tabular nature of this chapter to the deep RL methods that we'll learn.

The aim, as always, is to build a strong conceptual foundation and equip you with a flexible framework for reasoning about the core principles and the broader landscape in general.

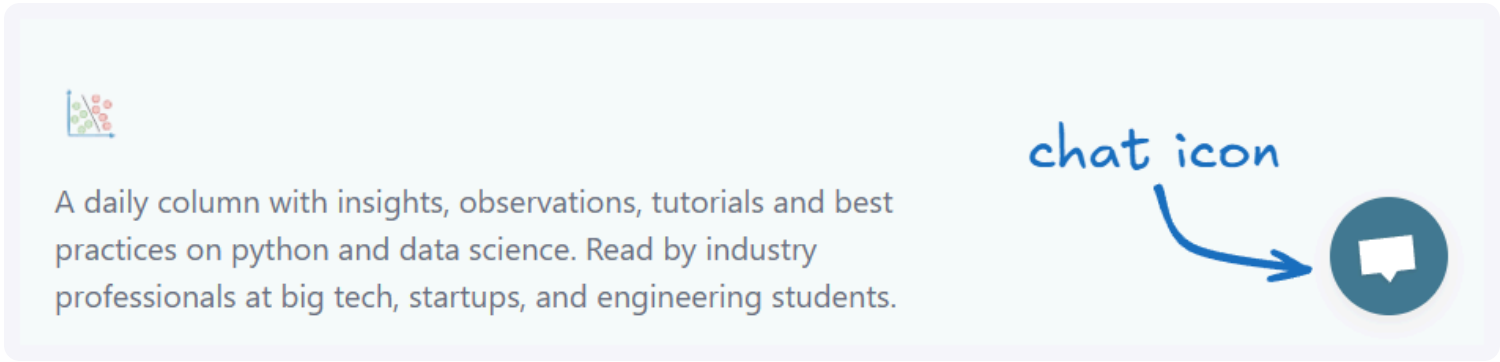
As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:



A daily colur
practices on
professionals

Published on May 17, 2026

Comments Share

Menu

Contact

FAQ

Previous
Bellman Equations and Dynamic Programming

Next
Function Approximation